

MatDataHub — Complete Developer Operations Manual

Version 1.0 | Prepared: 08/09/2026

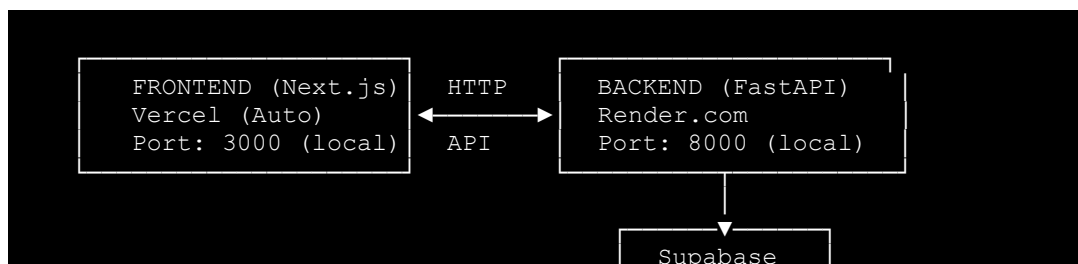
For: Kishan Kapoor (Admin & Lead Developer)

Table of Contents

1. [Project Overview & Architecture](#)
 2. [Folder Structure — What Each File Does](#)
 3. [Cleaning Up: Separating Old Streamlit from Live Project](#)
 4. [Setting Up Your Local Development Environment](#)
 5. [Daily Development Workflow](#)
 6. [How to Make Changes — Feature-by-Feature Guide](#)
 7. [Building, Testing & Deploying](#)
 8. [Common Errors & How to Fix Them](#)
 9. [Environment Variables Reference](#)
 10. [Quick Command Cheat Sheet](#)
-

1. Project Overview & Architecture

MatDataHub is a **full-stack web application** with two separate codebases that run independently:



	PostgreSQL
--	------------

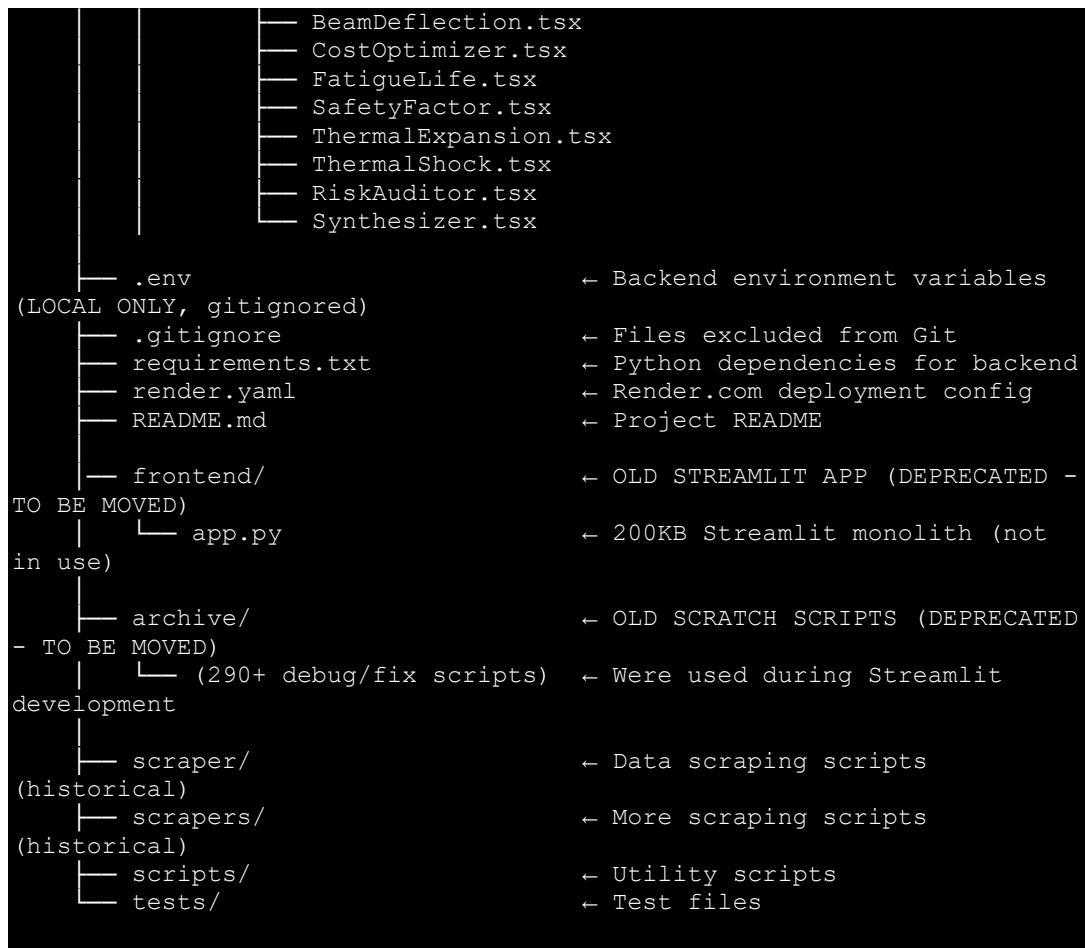
Layer	Technology	Hosted On	Git Hub Path
Frontend	Next.js 16 + React 19 + Tailwind CSS 4	Vercel	next-frontend/
Backend	Python FastAPI + SQLAlchemy	Render.com	app/
Database	PostgreSQL	Supabase	Cloud (no local files)
Old Frontend	Streamlit (DEPRECATED)	Not deployed	frontend/

2. Folder Structure — What Each File Does

Root Directory (MatDataHub/)

MatDataHub/	
├── app/	← BACKEND (FastAPI Python API)
│ ├── __init__.py	← Makes 'app' a Python package
│ └── main.py	← App entry point, CORS, startup
events	
│ └── database.py	← Supabase PostgreSQL connection
string	
│ └── models.py	← SQLAlchemy ORM models
(Material, User, Project, etc.)	
│ └── schemas.py	← Pydantic request/response
schemas	
│ └── auth.py	← JWT auth, session management,
TIER LIMITS	
│ └── workflows.py	← BOM Processor, CBAM calculator,
Substitution engine	
│ └── routers/	← API endpoint files (one per
feature)	
│ ├── materials.py	← GET /materials, compare,
similar, BOM analyze	
│ └── auth.py	← POST /register, /login, Google
OAuth	
│ ├── projects.py	← CRUD for project workspaces
│ └── ai.py	← AI Material Advisor (Grog API)

		admin.py	← Admin dashboard endpoints
		feedback.py	← Support tickets & feedback
		payments.py	← Razorpay integration
		account.py	← User profile management
		calculators.py	← Engineering calculator
endpoints			
	next-frontend/		← FRONTEND (Next.js React App)
	package.json		← NPM dependencies
	next.config.ts		← Next.js configuration
	tsconfig.json		← TypeScript configuration
	src/		
	lib/		
	api.ts		← Centralized API base URL
(IMPORTANT!)			
	app/		← Pages (each folder = one URL
route)			
		page.tsx	← / (Dashboard/Homepage)
		layout.tsx	← Root layout (sidebar wrapper)
		globals.css	← Global styles
		account/	
		page.tsx	← /account (Login, Register,
Profile, Billing)			
		admin/	
		page.tsx	← /admin (Admin Operations)
		ai/	
		page.tsx	← /ai (AI Material Advisor Chat)
		analytics/	
		page.tsx	← /analytics (Analytics Hub)
		cbam/	
		page.tsx	← /analytics/cbam (CBAM
Calculator)			
		compare/	
		page.tsx	← /analytics/compare (Side-by-
Side Compare)			
		substitution/	
		page.tsx	← /analytics/substitution (AI
Substitution)			
		synthesizer/	
		page.tsx	← /analytics/synthesizer
(Composite Synthesizer)			
		feedback/	
		page.tsx	← /feedback (Public Feedback
Wall)			
		materials/	
		page.tsx	← /materials (Browse Materials
Database)			
		[id]/	
		page.tsx	← /materials/123 (Material Detail
View)			
		projects/	
		page.tsx	← /projects (Project List)
		[id]/	
		page.tsx	← /projects/5 (Project Workspace)
		resources/	
		page.tsx	← /resources (Blogs, FAQs,
Support)			
		components/	← Reusable UI components
		Sidebar.tsx	← Main navigation sidebar
		MaterialSearchSelect.tsx	← Searchable material
dropdown			



Key Principle: "Which file controls which feature?"

Feature	Frontend File	Backend File
Login / Register / Google OAuth	account/page.tsx	routers/auth.py
Materials Database (Browse)	materials/page.tsx	routers/materials.py
Material Detail View	materials/[id]/page.tsx	routers/materials.py
Side-by-Side Compare	analytics/compare/page.tsx	routers/materials.py (compare endpoint)
AI Material Advisor	ai/page.tsx	routers/ai.py
Smart Substitution	analytics/substitution/page.tsx	routers/materials.py (similar endpoint)

Feature	Frontend File	Backend File
Composite Synthesizer	analytics/synthesizer/page.tsx	routers/materials.py (synthesize endpoint)
CBAM Calculator	analytics/cbam/page.tsx	workflows.py (BOMProcessor)
Project Workspace	projects/[id]/page.tsx	routers/projects.py
Admin Dashboard	admin/page.tsx	routers/admin.py
Feedback / Support	feedback/page.tsx, resources/page.tsx	routers/feedback.py
Payments (Razorpay)	account/page.tsx (upgrade buttons)	routers/payments.py
Engineering Calculators	Components in components/	routers/calculators.py
Sidebar Navigation	components/Sidebar.tsx	N/A
API Base URL Config	src/lib/api.ts	N/A

3. Cleaning Up: Separating Old Streamlit from Live Project

Your project currently has old Streamlit files mixed in. Here is how to separate them:

Step 1: Create a separate archive folder on your computer

```
C:\Users\KISHAN\Documents\Matdata\
├── MatDataHub/           ← LIVE PROJECT (keep this clean)
└── MatDataHub_Archive/   ← OLD FILES (move here)
```

Step 2: Move old files out (run these commands in PowerShell)

```
# Create archive folder
New-Item -ItemType Directory -Path
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive" -Force

# Move old Streamlit frontend
Move-Item "C:\Users\KISHAN\Documents\Matdata\MatDataHub\frontend"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\streamlit_frontend"

# Move old archive scripts
Move-Item "C:\Users\KISHAN\Documents\Matdata\MatDataHub\archive"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\old_scripts"

# Move old scraper files
Move-Item "C:\Users\KISHAN\Documents\Matdata\MatDataHub\scraper"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\scraper"
Move-Item "C:\Users\KISHAN\Documents\Matdata\MatDataHub\scrapers"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\scrapers"

# Move old database files
Move-Item "C:\Users\KISHAN\Documents\Matdata\MatDataHub\matdata.db"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\matdata.db"
Move-Item
"C:\Users\KISHAN\Documents\Matdata\MatDataHub\matdatahub_dev.db"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\matdatahub_dev.db"
"
Move-Item
"C:\Users\KISHAN\Documents\Matdata\MatDataHub\account_temp.txt"
"C:\Users\KISHAN\Documents\Matdata\MatDataHub_Archive\account_temp.txt"
```

Step 3: Remove from Git tracking

```
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub
git rm -r --cached frontend/
git rm -r --cached archive/
git rm -r --cached scraper/
git rm -r --cached scrapers/
git rm --cached matdata.db
git rm --cached matdatahub_dev.db
git rm --cached account_temp.txt
git commit -m "chore: remove deprecated Streamlit files and old
scripts from repo"
git push origin main
```

Step 4: Update .gitignore

Add these lines to your `.gitignore`:

```
frontend/  
archive/  
scraper/  
scrapers/  
matdata.db  
matdatahub_dev.db  
account_temp.txt  
*.db  
__pycache__/  
.venv/  
venv/  
node_modules/  
.next/  
.env
```

After cleanup, your clean project structure will be:

```
MatDataHub/  
├── app/           ← Backend (Python FastAPI)  
├── next-frontend/ ← Frontend (Next.js React)  
├── tests/         ← Test files  
├── scripts/       ← Utility scripts  
├── .env           ← Local env vars (gitignored)  
├── .gitignore  
├── requirements.txt  
├── render.yaml  
└── README.md
```

4. Setting Up Your Local Development Environment

Prerequisites

Install these on your computer:

1. **Node.js** (v20+) — <https://nodejs.org>
2. **Python** (v3.10+) — <https://python.org>
3. **Git** — <https://git-scm.com>
4. **VS Code** (recommended editor) — <https://code.visualstudio.com>

First-Time Setup

Clone the project:

```
cd C:\Users\KISHAN\Documents\Matdata
git clone https://github.com/kk873807/MatDataHub.git
cd MatDataHub
```

Setup Backend:

```
# Create Python virtual environment
python -m venv .venv

# Activate it (run this EVERY TIME you open a new terminal)
.venv\Scripts\Activate

# Install Python dependencies
pip install -r requirements.txt
```

Setup Frontend:

```
cd next-frontend
npm install
cd ..
```

Create .env file:

Create a file called `.env` in the `MatDataHub/` root with:

```
DATABASE_URL=postgresql://postgres:xxxxx:password@aws-0-ap-south-1.pooler.supabase.com:6543/postgres
GROQ_API_KEY=gsk_xxxxxxxxxxxxxx
ADMIN_SECRET=your_admin_secret
GOOGLE_CLIENT_ID=870854614604-xxxxx.apps.googleusercontent.com
```

```
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxx  
FRONTEND_URL=https://matdatahub.vercel.app  
OAUTH_SESSION_SECRET=your_random_string_here
```

5. Daily Development Workflow

Starting the app locally:

Terminal 1 — Backend:

```
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub  
.venv\Scripts\Activate  
uvicorn app.main:app --reload --port 8000
```

Backend will run at: `http://localhost:8000`

API docs at: `http://localhost:8000/docs`

Terminal 2 — Frontend:

```
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub\next-frontend  
npm run dev
```

Frontend will run at: `http://localhost:3000`

IMPORTANT: For local development, temporarily change `next-frontend/src/lib/api.ts` to:

```
export const API_BASE = "http://localhost:8000";
```

Change it back before pushing to GitHub!

The Edit → Test → Push cycle:

```
1. Make your code change in VS Code
```

2. Save the file (Ctrl+S)
3. Check the browser — Next.js auto-refreshes (hot reload)
4. If it works, build the frontend to check for errors:
`cd next-frontend`
`npm run build`
5. If build succeeds, commit and push:
`cd ..`
`git add .`
`git commit -m "describe what you changed"`
`git push origin main`
6. Vercel auto-deploys frontend in ~60 seconds
7. Render auto-deploys backend in ~3-5 minutes

6. How to Make Changes — Feature-by-Feature Guide

Changing the UI/look of a page

1. Find the page file in `next-frontend/src/app/<pagename>/page.tsx`
2. Edit the JSX (HTML-like code) and Tailwind CSS classes
3. Save → browser auto-refreshes
4. Run `npm run build` to check for errors before pushing

Example — Change the dashboard title:

Open `next-frontend/src/app/page.tsx`, find the `<h1>` tag, change text.

Adding a new material property to the database

1. Add the column in `app/models.py` (SQLAlchemy model)
2. Add the field in `app/schemas.py` (Pydantic schema)
3. Run a database migration on Supabase (SQL ALTER TABLE)
4. Update the frontend display in `materials/[id]/page.tsx`

Adding a new API endpoint

1. Create or edit a file in `app/routers/`
2. Register it in `app/main.py` with `app.include_router(...)`
3. Test at `http://localhost:8000/docs`

Adding a new frontend page

1. Create folder: `next-frontend/src/app/newpage/`
2. Create file: `next-frontend/src/app/newpage/page.tsx`
3. Add `"use client";` at the top
4. Add a sidebar link in `next-frontend/src/components/Sidebar.tsx`

Changing the sidebar

Edit: `next-frontend/src/components/Sidebar.tsx`

Changing the API URL

Edit: `next-frontend/src/lib/api.ts`

- For local dev: `http://localhost:8000`
- For production: `https://matdatahub-api.onrender.com`

7. Building, Testing & Deploying

Frontend Build (must pass before pushing)

```
cd next-frontend
npm run build
```

If it says "✓ Generating static pages (16/16)" — you're good!
If it shows errors — fix them before pushing.

Common build errors:

Error	Meaning	Fix
Cannot find name 'X'	Forgot to import something	Add the import at top of file
Type 'X' is not assignable	TypeScript type mismatch	Check your variable types
useSearchParams() should be wrapped in Suspense	Next.js 16 requirement	Use <code>window.location.search</code> instead
Module not found	Package not installed	Run <code>npm install packagename</code>

Deploying to Production

Frontend (Vercel) — AUTOMATIC:

- Every `git push origin main` triggers Vercel auto-deploy
- Vercel reads from `next-frontent/` subdirectory
- Takes ~60 seconds
- Check status: <https://vercel.com/dashboard>

Backend (Render) — AUTOMATIC:

- Every `git push origin main` triggers Render auto-deploy
- Takes ~3-5 minutes (free tier has cold starts)
- Check status: <https://dashboard.render.com>

Database (Supabase) — MANUAL:

- Database changes (new columns, tables) require manual SQL
- Go to <https://supabase.com/dashboard>
- Click your project → SQL Editor → Run your ALTER TABLE commands

8. Common Errors & How to Fix Them

"Error 400: redirect_uri_mismatch" (Google OAuth)

Fix: Go to Google Cloud Console → APIs & Services → Credentials → OAuth 2.0 Client → Add the correct redirect URI.

"CORS error" in browser console

Fix: Check `app/main.py` → `allow_origins` list. Make sure your frontend URL is listed.

"429 Too Many Requests"

Fix: This is the rate limiter. Wait 60 seconds, or increase `MAX_REQS_PER_MIN` in `app/routers/materials.py`.

"Module not found: react-markdown"

Fix: `cd next-frontend && npm install react-markdown remark-gfm`

Backend crashes on Render

Fix: Check Render logs. Usually missing environment variables. Set them in Render Dashboard → Environment.

"localStorage is not defined"

Fix: Wrap `localStorage` calls in `typeof window !== "undefined"` check. This is a Next.js server-side rendering issue.

9. Environment Variables Reference

Backend (.env file — local only)

Variable	Where to Set	Description
DATABASE_URL	.env + Render Dashboard	Supabase PostgreSQL connection string
GROQ_API_KEY	.env + Render Dashboard	Groq API key for AI features
ADMIN_SECRET	.env + Render Dashboard	Secret password for /admin page
GOOGLE_CLIENT_ID	.env + Render Dashboard	Google OAuth client ID
GOOGLE_CLIENT_SECRET	.env + Render Dashboard	Google OAuth client secret
FRONTEND_URL	.env + Render Dashboard	https://matdatahub.vercel.app
OAuth_SESSION_SECRET	.env + Render Dashboard	Random string for OAuth sessions
RAZORPAY_KEY_ID	Render Dashboard only	Razorpay API Key (when ready)
RAZORPAY_KEY_SECRET	Render Dashboard only	Razorpay Secret (when ready)

Frontend (Vercel Environment Variables)

Variable	Value
NEXT_PUBLIC_API_URL	https://matdatahub-api.onrender.com

10. Quick Command Cheat Sheet

```
# ===== DAILY COMMANDS =====

# Start backend locally
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub
.venv\Scripts\Activate
uvicorn app.main:app --reload --port 8000

# Start frontend locally (separate terminal)
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub\next-frontend
npm run dev

# ===== BUILD & DEPLOY =====

# Build frontend (check for errors)
cd next-frontend
npm run build

# Push all changes to production
cd C:\Users\KISHAN\Documents\Matdata\MatDataHub
git add .
git commit -m "your message here"
git push origin main

# ===== GIT COMMANDS =====

# Check what files changed
git status

# See recent commits
git log -n 5 --oneline

# Undo last commit (keep changes)
git reset --soft HEAD~1

# Pull latest changes
git pull origin main

# ===== PACKAGE MANAGEMENT =====

# Install a new NPM package (frontend)
cd next-frontend
npm install package-name

# Install a new Python package (backend)
pip install package-name
pip freeze > requirements.txt

# ===== DATABASE =====

# Access Supabase SQL Editor
# Go to: https://supabase.com/dashboard → SQL Editor

# Example: Add a new column
# ALTER TABLE materials ADD COLUMN new_property FLOAT;
```

```
# ===== TROUBLESHOOTING =====

# Clear Next.js cache
cd next-frontend
Remove-Item -Recurse -Force .next
npm run build

# Check TypeScript errors without building
cd next-frontend
npx tsc --noEmit

# Test backend API
# Open browser: http://localhost:8000/docs
```

Summary: The 5-Minute Decision Tree

"I want to change how something LOOKS"

→ Edit the `.tsx` file in `next-frontend/src/app/`

"I want to change how something WORKS (data, logic)"

→ Edit the `.py` file in `app/routers/`

"I want to add a new PAGE"

→ Create folder + `page.tsx` in `next-frontend/src/app/`, add sidebar link

"I want to add a new API"

→ Create/edit file in `app/routers/`, register in `app/main.py`

"I want to change the DATABASE"

→ Edit `app/models.py`, then run SQL on Supabase

"I want to change PRICING or TIER LIMITS"

→ Edit `TIER_LIMITS` in `app/auth.py` (line 88)

"Something is BROKEN"

→ Check browser console (F12) for frontend errors, check Render logs for backend errors

Document prepared for MatDataHub v1.0
GitHub: <https://github.com/kk873807/MatDataHub>
Live: <https://matdatahub.vercel.app>